

```
gayathri@Gayathri: ~/pipe_ls x + v
gayathri@Gayathri:~$ mkdir producer_consumer
gayathri@Gayathri:~$ cd producer_consumer
gayathri@Gayathri:~/producer_consumer$ pwd
/home/gayathri/producer_consumer
gayathri@Gayathri:~/producer_consumer$ nano producer_consumer.c
gayathri@Gayathri:~/producer_consumer$ ls
producer_consumer.c
gayathri@Gayathri:~/producer_consumer$ gcc producer_consumer.c -o producer_consumer
gayathri@Gayathri:~/producer_consumer$ ./producer_consumer

Consumer finished.
Data consumed: 100000
Sum of consumed data: 5000050000

Producer finished.
Data produced: 100000
Communication time: 0.025012 seconds
Communication rate: 3998080.92 values/second
gayathri@Gayathri:~/producer_consumer$ cd ..
gayathri@Gayathri:~$ pwd
/home/gayathri
gayathri@Gayathri:~$ mkdir pipe_ls_grep
gayathri@Gayathri:~$ cd pipe_ls_grep
gayathri@Gayathri:~/pipe_ls_grep$ nano ls_grep.c
gayathri@Gayathri:~/pipe_ls_grep$ ls
ls_grep.c
gayathri@Gayathri:~/pipe_ls_grep$ gcc ls_grep.c -o ls_grep
gayathri@Gayathri:~/pipe_ls_grep$ ./ls_grep
-rw-r--r-- 1 gayathri gayathri 1183 Aug 30 06:59 ls_grep.c

Command executed successfully.
gayathri@Gayathri:~/pipe_ls_grep$ touch test.c
touch hello.txt
touch program.c
touch notes.txt
```

```
gayathri@Gayathri: ~/pipe_ls x + v
gayathri@Gayathri:~/producer_consumer$ gcc producer_consumer.c -o producer_consumer
gayathri@Gayathri:~/producer_consumer$ ./producer_consumer

Consumer finished.
Data consumed: 100000
Sum of consumed data: 5000050000

Producer finished.
Data produced: 100000
Communication time: 0.025012 seconds
Communication rate: 3998080.92 values/second
gayathri@Gayathri:~/producer_consumer$ cd ..
gayathri@Gayathri:~$ pwd
/home/gayathri
gayathri@Gayathri:~$ mkdir pipe_ls_grep
gayathri@Gayathri:~$ cd pipe_ls_grep
gayathri@Gayathri:~/pipe_ls_grep$ nano ls_grep.c
gayathri@Gayathri:~/pipe_ls_grep$ ls
ls_grep.c
gayathri@Gayathri:~/pipe_ls_grep$ gcc ls_grep.c -o ls_grep
gayathri@Gayathri:~/pipe_ls_grep$ ./ls_grep
-rw-r--r-- 1 gayathri gayathri 1183 Aug 30 06:59 ls_grep.c

Command executed successfully.
gayathri@Gayathri:~/pipe_ls_grep$ touch test.c
touch hello.txt
touch program.c
touch notes.txt
gayathri@Gayathri:~/pipe_ls_grep$ ./ls_grep
-rw-r--r-- 1 gayathri gayathri 1183 Aug 30 06:59 ls_grep.c
-rw-r--r-- 1 gayathri gayathri 0 Aug 30 07:00 program.c
-rw-r--r-- 1 gayathri gayathri 0 Aug 30 07:00 test.c

Command executed successfully.
gayathri@Gayathri:~/pipe_ls_grep$ |
```

1st used code:-

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <time.h>

```
#define BUFFER_SIZE 10
```

```
int main()
```

```
{
```

```
    int pipefd[2];
```

```
    pid_t pid;
```

```
    if (pipe(pipefd) == -1)
```

```
    {
```

```
        perror("pipe");
```

```
        return 1;
```

```
    }
```

```
    pid = fork();
```

```
    if (pid < 0)
```

```
    {
```

```
        perror("fork");
```

```
        return 1;
```

```
    }
```

```
    if (pid > 0)
```

```
    {
```

```
        /* Parent - Producer */
```

```
        close(pipefd[0]);
```

```
        int data;
```

```
        int count = 100000;
```

```
clock_t start, end;
```

```
start = clock();
```

```
for (int i = 1; i <= count; i++)
```

```
{
```

```
    data = i;
```

```
    if (write(pipefd[1], &data, sizeof(data)) == -1)
```

```
    {
```

```
        perror("write");
```

```
        break;
```

```
    }
```

```
}
```

```
close(pipefd[1]);
```

```
wait(NULL);
```

```
end = clock();
```

```
double time_taken =
```

```
    (double)(end - start) / CLOCKS_PER_SEC;
```

```
printf("\nProducer finished.\n");
```

```
printf("Data produced: %d\n", count);
```

```
printf("Communication time: %.6f seconds\n", time_taken);
```

```
if (time_taken > 0)
```

```
{
```

```
    printf("Communication rate: %.2f values/second\n",
```

```

        count / time_taken);

    }

}

else

{

    /* Child - Consumer */

    close(pipefd[1]);

    int data;

    long long sum = 0;

    int count = 0;

    while (read(pipefd[0], &data, sizeof(data)) > 0)
    {
        sum += data;

        count++;
    }

    close(pipefd[0]);

    printf("\nConsumer finished.\n");

    printf("Data consumed: %d\n", count);

    printf("Sum of consumed data: %lld\n", sum);

}

return 0;

}

```

2nd used code:-

```
#include <stdio.h>
```

```
#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int pipefd[2];
```

```
    if (pipe(pipefd) == -1)
```

```
    {
```

```
        perror("pipe");
```

```
        return 1;
```

```
    }
```

```
    pid_t pid1, pid2;
```

```
    /* Create first child */
```

```
    pid1 = fork();
```

```
    if (pid1 < 0)
```

```
    {
```

```
        perror("fork");
```

```
        return 1;
```

```
    }
```

```
    if (pid1 == 0)
```

```
    {
```

```
        /* First child: execute ls -l */
```

```
        close(pipefd[0]);
```

```
dup2(pipefd[1], STDOUT_FILENO);
```

```
close(pipefd[1]);
```

```
execlp("ls", "ls", "-l", NULL);
```

```
perror("execlp ls");
```

```
exit(1);
```

```
}
```

```
/* Create second child */
```

```
pid2 = fork();
```

```
if (pid2 < 0)
```

```
{
```

```
    perror("fork");
```

```
    return 1;
```

```
}
```

```
if (pid2 == 0)
```

```
{
```

```
    /* Second child: execute grep ".c" */
```

```
close(pipefd[1]);
```

```
dup2(pipefd[0], STDIN_FILENO);
```

```
close(pipefd[0]);
```

```
execlp("grep", "grep", ".c", NULL);
```

```
    perror("execlp grep");  
    exit(1);  
}  
  
/* Parent process */  
  
close(pipefd[0]);  
close(pipefd[1]);  
  
waitpid(pid1, NULL, 0);  
waitpid(pid2, NULL, 0);  
  
printf("\nCommand executed successfully.\n");  
  
return 0;  
}
```